Name: Angel A. Orillana

Role: Client 1 Creator and Backend Contributor (Email and Password validation)

I am responsible for creating one of the required clients for this activity, specifically client1.py. I created it using Python and implemented a menu system. When the program runs, users are greeted with a welcome message and a main menu with three options: 1. Register, 2. Login, and 3. Exit.

If the user chooses Register, they are asked to input a username, email, and password. If the username or email is already taken, the API returns a message and the user is prompted to try again. Whatever the input, the client will always return a message, whether to inform that the input is incorrect or confirm that the action is successful. The same behavior applies to Login. After a successful login, a welcome message is displayed, followed by a user menu with four options: 1. Register Shop, 2. View Shop, 3. Update Shop, and 4. Logout.

In Register Shop, the user is only allowed to register one shop. If they already have one, the system will return a message and prevent them from creating another. It also checks the response to confirm if the shop name is already taken to prevent duplication. For View Shop, the system retrieves the shop details from the API. If the user already has a shop, it will display the shop information such as username, shop name, and shop status. If no shop exists yet, a message will indicate that no shop is found. For Update Shop, the user is allowed to change their existing shop name. Before updating, the system checks the current shop details. If the user inputs a new shop name that already exists or is the same as the current one, the response message will inform the user of the issue. If the update is successful, the updated information will be returned and displayed in the client. For Logout, the user will be redirected back to the main menu. If the user chooses Exit from the main menu, the system will close.

While developing the client, I encountered challenges in handling API responses and making sure the program guided users properly. At first, some responses were not displayed clearly, which made the flow confusing. For instance, the username and email error message is merged into one, which makes it hard for me to know which of my input is incorrect. I solved this by carefully checking the status codes and adjusting the service.php and how the client printed messages. Another challenge was designing the menu system so that users could navigate smoothly between Register, Login, and the shop functions without getting stuck. I tested different inputs and repeated runs to ensure the client handled invalid choices and errors gracefully.

With that, I also added restrictions on the email domain, allowing only @gmail.com,

@bicol-u.edu.ph, and @outlook.com. For the password, I implemented a rule requiring a minimum of 8 characters, which can be a combination of letters, special characters, and digits, but must not be purely letters, numbers, or symbols. Overall, all messages displayed in the client are based on responses from the API, depending on the status codes returned. This ensures that validation and system rules are handled consistently on the backend, while the client focuses on displaying and handling user interaction.